

Examen Intelligence Artificielle 1

Session 1 : 11 décembre 2025 (Correction)

Durée : 1 h 30

Matériel autorisé :

Une feuille A4 recto-verso de notes personnelles, une calculatrice simple (pas de téléphone portable).

Barème (indicatif) : 3+5+5+2

Les exercices sont indépendants. Vous pouvez les traiter dans **l'ordre de votre choix**.

Lisez bien les questions. Il vous est demandé d'être clair, concis, mais de **justifier vos résultats**.

I) Questions générales

Exercice 1 Questions générales

1. Pourquoi les couches cachées d'un perceptron multicouche doivent-elles être activées par une fonction non linéaire ?

Correction : Sans les activations non linéaires, un perceptron multicouche aurait les mêmes capacités qu'un perceptron monocouche (les sorties z_i ne seraient que des combinaisons linéaires des entrées x_i).

2. Donnez deux exemples de fonctions d'activation.

Correction : Tangente hyperbolique et ReLU.

3. Dans le cadre des *algorithmes génétiques*, indiquez si les opérateurs de sélection, de recombinaison et de mutation agissent respectivement dans le sens de l'exploration ou de l'exploitation. Justifiez brièvement.

Correction : L'opérateur de sélection favorise les meilleures solutions courantes, c.-à-d. agit du côté de l'exploitation, alors que les opérateurs de recombinaison tendent à diversifier les génotypes, c.-à-d. agissent dans le sens de l'exploration.

II) Recherche dans un espace d'états

Exercice 2 Tortues solidaires

Il est bien connu qu'une tortue sur le dos ne peut se retourner toute seule. Ce que l'on sait moins, c'est que deux tortues côte à côte peuvent s'entraider et se retourner toutes les deux (si, si). Cette délicate opération n'est possible qu'entre deux individus adjacents. Rien n'empêche d'ailleurs que deux tortues sur leurs pattes se remettent conjointement sur le dos. Et évidemment, si une tortue est sur ses pattes tandis que sa voisine est sur son dos, la manœuvre aurait pour effet d'inverser leurs situations.

Le problème à résoudre est de trouver une succession de retournements qui amènerait quatre tortues alignées à être toutes sur leurs pattes, partant d'une certaine configuration initiale (voir figure 1).

1. Proposez un espace d'états $\mathcal{S}$ pour ce problème. Vous expliquerez quelle est l'information encodée par un état, et exprimerez $|\mathcal{S}|$.

Correction : Un état peut être représenté par un quadruplet de valeurs binaires, dont le i -ème élément indique la situation de la i -ème tortue : 0 si elle est sur le dos, 1 si elle est sur pattes.

- Espace d'états : $\mathcal{S} = \{0, 1\}^4$, $|\mathcal{S}| = 2^4$

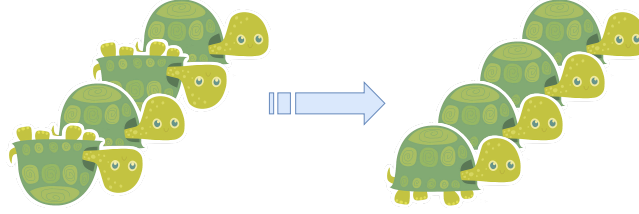


FIGURE 1 – La situation initiale (à gauche) et la configuration but (à droite) pour le problème des tortues.

2. Identifiez l'état initial s_{init} , et l'ensemble des états finaux $\mathcal{F}$ (en vous référant à la figure 1).

Correction :

- État initial : $s_{\text{init}} = (0, 1, 0, 1)$
- États finaux : $\mathcal{F} = \{(1, 1, 1, 1)\}$

3. Définissez un unique opérateur pour le retournement de deux tortues adjacentes (peu importe leur situation), pour lequel on prendra un coût constant de 1.

Correction :

- Opérateurs :

Nom	Paramètres	Préconditions sur un état $s \in \mathcal{S}$	Effet	Coût
retourne	$i \in \llbracket 0; 2 \rrbracket$	$\emptyset$	• $s_i \leftarrow 1 - s_i$ • $s_{i+1} \leftarrow 1 - s_{i+1}$	1

4. On considère les fonctions heuristiques suivantes :

- h_1 : Nombre de tortues sur leur dos.
- h_2 : Nombre de tortues sur leur dos divisé par deux.

Pour chacune, précisez si elle est minorante. Laquelle est la meilleure ? Justifiez.

Remarque : On admettra que le nombre de tortues sur leur dos est toujours pair.

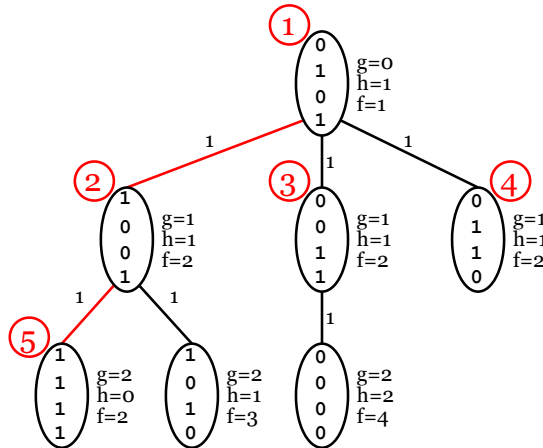
Correction : L'opérateur retourne, de coût 1, n'agit que sur deux tortues à la fois. h_2 est donc minorante, de façon évidente. En revanche h_1 ne l'est pas, puisque, par exemple :

$$h_1((0, 0, 1, 1)) = 2 > 1 = h^*((0, 0, 1, 1))$$

h_2 est une meilleure heuristique, dans la mesure où elle nous permettra d'appliquer A^* .

5. Appliquez l'algorithme A^* avec h_2 .

Correction :



Chemin optimal : $(0, 1, 0, 1) \rightarrow (1, 0, 0, 1) \rightarrow (1, 1, 1, 1)$.

III) Algorithmes de jeux

Exercice 3 AlphaBeta à profondeur incrémentale

On se propose d'appliquer l'algorithme AlphaBeta à l'arbre de jeu décrit implicitement dans le tableau 1, en incrémentant de façon itérative la profondeur d'exploration maximale $p_{\max}$.

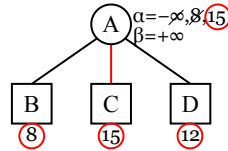
Pour chacune des questions ci-dessous, on supposera que les fils d'un nœud AMI sont à parcourir par ordre de valeurs décroissantes, et que les fils d'un nœud ENNEMI sont à parcourir par ordre de valeurs croissantes, en se référant aux valeurs trouvées à la précédente question. On considérera que les fils coupés lors de l'itération précédente sont à parcourir en dernier. Enfin, pour départager les fils non évalués durant l'itération précédente (car coupés ou au delà de la profondeur maximale), on suivra l'ordre alphabétique.

nœud n	A	B	C	D	E	F	G	H	I	J
$f(n)$	13	8	15	12	7	55	19	28	17	11
succ(n)	B C D	E F G	H I J	K L	M N	O P	Q R S	T U	V W	X Y
nœud n	K	L	M	N	O	P	Q	R	S	T
$f(n)$	14	15	10	11	20	13	10	11	6	14
succ(n)	Z AA	AB AC	-	-	-	-	-	-	-	-
nœud n	U	V	W	X	Y	Z	AA	AB	AC	
$f(n)$	30	18	19	12	10	14	12	15	8	

TABLE 1 – Évaluation heuristique (f) et successeurs (succ) des différents nœuds d'un arbre de jeu (A est la racine).

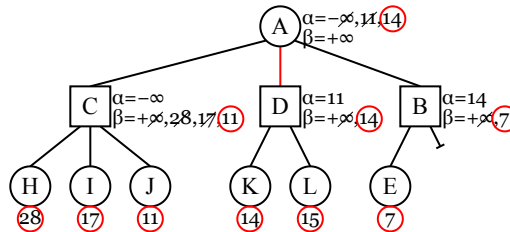
1. Appliquez AlphaBeta pour $p_{\max} = 1$.

Correction :



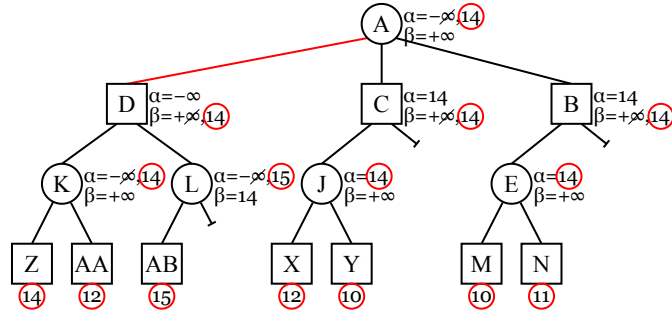
2. Appliquez AlphaBeta pour $p_{\max} = 2$.

Correction :



3. Appliquez AlphaBeta pour $p_{\max} = 3$.

Correction :



4. Expliquez pourquoi AlphaBeta à profondeur incrémentale peut être utile dans un contexte où le temps réservé à la préparation d'un coup est limité.

Correction : Un programme utilisant AlphaBeta à profondeur incrémentale est capable de donner une réponse de coup à jouer à n'importe quel moment : l'itération initiale, peu coûteuse, donne une première réponse, qui sera améliorée au fil des itérations, jusqu'à ce que le temps imparti soit écoulé.

IV) Programmation par contraintes

Exercice 4 Carré latin

Un carré latin d'ordre n est un tableau de taille $n \times n$, contenant exactement n valeurs distinctes (p. ex. les entiers de 1 à n), dont chaque ligne et chaque colonne ne contient qu'un seul exemplaire. Les matrices ci-dessous sont notamment des exemples de carrés latins d'ordre 3 :

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \\ 3 & 1 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 1 & 3 \\ 3 & 2 & 1 \\ 1 & 3 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 3 & 2 \\ 3 & 2 & 1 \\ 2 & 1 & 3 \end{bmatrix}$$

1. Formalisez (en définissant variables, domaines, contraintes) le problème de constitution d'un carré latin d'ordre 3, rempli avec les entiers de 1 à 3.

Correction :

- Variables :
 $\mathcal{X} = \{X_{ij} | i, j \in \llbracket 0; 2 \rrbracket\}$
 X_{ij} : valeur de la case ligne i , colonne j .
- Domaines :
 $\mathcal{D} = \{\mathcal{D}_{ij} | i, j \in \llbracket 0; 2 \rrbracket\}$
 $\mathcal{D}_{ij} = \llbracket 1; 3 \rrbracket$
- Contraintes :
 $\mathcal{C} = \{C_{L1}, C_{L2}, C_{L3}, C_{C1}, C_{C2}, C_{C3}\}$
 $C_{Li} : \nexists j, j' \in \llbracket 0; 2 \rrbracket, j \neq j' \text{ t.q. } X_{ij} = X_{ij'}$
 $C_{Cj} : \nexists i, i' \in \llbracket 0; 2 \rrbracket, i \neq i' \text{ t.q. } X_{ij} = X_{i'j}$

2. Donner le nom d'un algorithme vu en cours qui conviendrait pour trouver une solution au CSP que vous venez de définir.

Correction : Génère et teste, ou retour sur trace simple.

Remarque : Les algorithmes de résolution avec anticipation vus en cours – *forward checking*, *partial lookahead*, *full lookahead* – pourraient être appliqués, à condition d'avoir formalisé le problème avec des contraintes binaires, p. ex. :

$$\mathcal{C} = \{C_{ij-kl} | i, j, k, l \in \llbracket 0; 2 \rrbracket \text{ et } i = k \oplus j = l\}$$

$$C_{ij-kl} : X_{ij} \neq X_{kl}$$